

Universidade do Minho

Interação entre os serviços AIDA e PACS com utilização do Mirth Connect

Realizado no âmbito da Unidade Curricular de
Aplicações Informáticas em Engenharia Biomédica

Discentes:

Madalena Passos (pg54023)

Mariana Almeida (pg54062)

Mariana Ribeiro (pg5061)

Mónica Martins (a95918)

Docente: José Machado

abril de 2024

Índice

Introdução	2
Serviços implementados	2
Arquitetura	3
Canais Mirth.....	4
Programas Python	4
Estruturas em JSON	5
AIDA.py	6
PACS.py	8
Conclusão.....	11

Introdução

No âmbito da Unidade Curricular de Aplicações Informáticas em Engenharia Biomédica foi proposta a emulação de um sistema de comunicação entre dois serviços hospitalares, com recurso ao Mirth Connect.

Neste projeto pretendia-se, então, o desenvolvimento de um sistema para troca de mensagens no contexto da engenharia biomédica, utilizando o formato HL7 (Health Level Seven). HL7 é um conjunto de normas internacionais para transferência de informações clínicas e administrativas entre sistemas de informação de saúde, desempenhando um papel muito importante na interoperabilidade e comunicação eficaz entre diferentes serviços de saúde.

No presente relatório, será detalhado o sistema desenvolvido, nomeadamente entre dois serviços, AIDA e PACS, com funcionalidades específicas relacionadas à gestão de pedidos e exames médicos. Para isso, foram desenvolvidos dois programas em Python e configurados quatro canais Mirth, proporcionando uma arquitetura robusta para a troca de informações entre os serviços.

Resumidamente, os objetivos deste trabalho foram os seguintes:

- . Criação de canais Mirth que permitissem o fluxo de mensagens HL7 na versão 2.5;
- . Desenvolvimento de dois programas em Python, de modo a simular as estruturas AIDA e PACS de um serviço hospitalar;
- . Implementação de menus com várias funcionalidades que permitissem fazer e cancelar pedidos, realizar exames, entre outros;
- . Guardar o estado dos pedidos numa estrutura JSON.

Serviços implementados

Como referido anteriormente, foram considerados dois serviços (AIDA e PACS) com as seguintes funcionalidades:

- . AIDA:
 - Ver todos os pedidos: permite consultar informação relativa a todos os pedidos efetuados para todos os utentes;
 - Ver pedido: dado o ID de um determinado paciente (PID) e o ID de um pedido relativo ao mesmo, permite consultar informação acerca do mesmo (Tipo de pedido, data de transação, Status, Order Control, segmento OBX, entre outros);
 - Fazer pedido: permite, dadas as informações de um utente, fazer um novo pedido para o mesmo;

- Cancelar pedido: permite alterar o *status* de um pedido existente para “CA” (cancelado);

. PACS:

- Ver todos os pedidos: funcionalidade igual à descrita no serviço AIDA;
- Cancelar pedido: semelhante à funcionalidade do serviço AIDA;
- Ver pedidos pendentes: permite, dado um PID, listar os pedidos do paciente cujo *status* seja diferente de “CM” ou “CA”;
- Realização de exame: permite, para um dado PID e ID de pedido, escrever o respetivo exame;
- Realização de relatório: dado um PID e ID de pedido, permite escrever o respetivo relatório ou nome do ficheiro.

Posto isto, foi necessário desenvolver dois programas em Python para abrir os ficheiros que continham as mensagens HL7 enviadas e recebidas, processar as mensagens e guardar a informação atualizada acerca das mesmas, numa estrutura JSON. Para além disso, foi necessário criar canais Mirth para garantir a interação e troca de mensagens entre serviços.

Arquitetura

Tendo em conta os serviços referidos, foi necessário garantir um fluxo de comunicação. Para tal, foram configurados quatro canais Mirth, que asseguravam a comunicação através de quatro pastas. Na Figura 1 está representada a arquitetura do sistema desenvolvido.

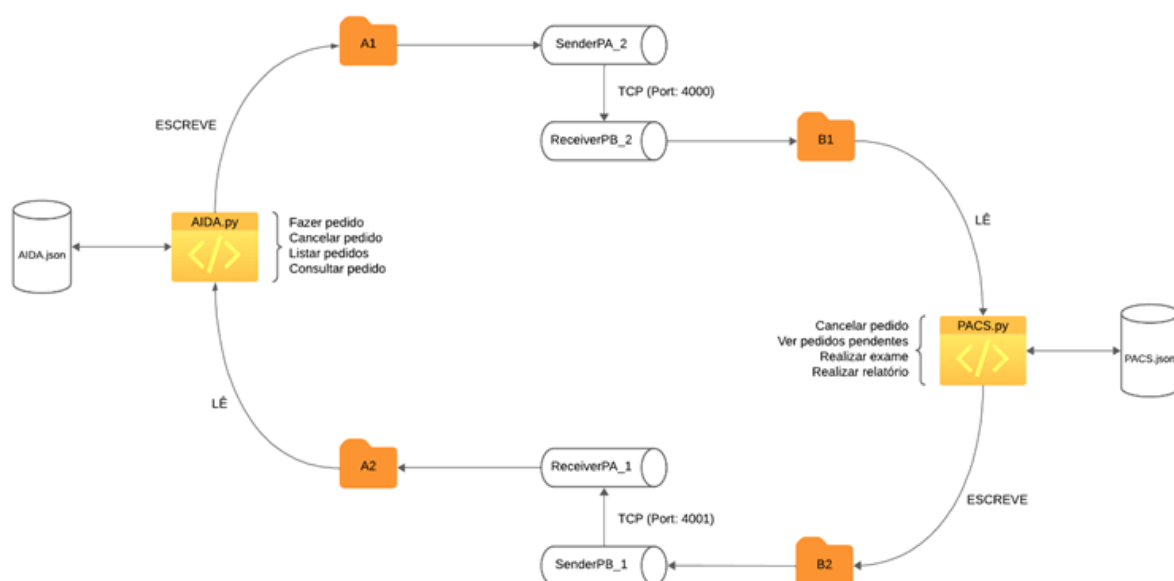


Figura 1- Arquitetura do sistema desenvolvido.

Canais Mirth

Uma vez que o objetivo do trabalho consiste na interação e troca de mensagens entre dois serviços, são necessários 2 canais para cada um deles:

- . Sender: Este canal deverá ser capaz de ler ficheiros em formato HL7 presentes numa diretoria e enviá-los para o outro serviço, através de uma conexão TCP. Assim, os tipos de conector serão “File Reader” (*source*) e “TCP writer” (*destination*);
- . Receiver: Este canal, como o nome indica, deve ser ativo sempre que um ficheiro em formato HL7 é enviado de outro serviço (por TCP), para que seja possível guardá-lo. Posto isto, o conector *source* será do tipo “TCP listener” e o *destination* será “File writer”.

Definiu-se que o serviço PACS iria escrever e colocar as mensagens (ou ficheiros) HL7 numa diretoria chamada “B2”. Por outro lado, definiu-se que o serviço AIDA iria colocar os ficheiros HL7 na diretoria “A1”.

Posto isto, falta apenas definir quais as diretorias onde os canais do tipo “Receiver” irão colocar os ficheiros que recebem. Assim, definiu-se que os canais “receiver” dos serviços AIDA e PACS colocariam os ficheiros recebidos nas diretorias “A2” e “B1”.

Em suma, são necessários 2 canais mirth para cada serviço:

- . Um canal que espera que sejam colocados ficheiros HL7 (file reader) nas diretorias “A1” ou “B2” e, quando isso acontece, envia os ficheiros por TCP (TCP writer nas portas 4000 ou 4001);
- . Um canal que fica à “escuta” nas portas referidas (TCP listener) e, quando recebe algum ficheiro, coloca-o na diretoria “A2” e “B1” (file writer).

Tendo a estrutura definida, resta desenvolver os programas Python de modo a garantir que os serviços sejam capazes de abrir e processar ficheiros com mensagens HL7 recebidas e armazenar o estado atual dos pedidos (tanto quando se enviam como quando se recebem mensagens).

Programas Python

De seguida serão explicados, de forma sucinta, os programas desenvolvidos para cada um dos serviços.

Estruturas em JSON

De forma a armazenar todos os registos médicos de pacientes, foi definida a seguinte estrutura em JSON, utilizada pelos sistemas AIDA e PACS. Ambos os sistemas utilizam a mesma estrutura de dados JSON, embora cada um tenha o seu próprio ficheiro JSON para armazenar as suas informações.

```
{
  PID1 : {"Nome": nome do paciente PID1,
    "Data de nascimento": data de nascimento do paciente PID1,
    "Genero": género do paciente PID1,
    "Pedidos": { IDpedido1: {
      "Tipo de pedido": tipo do pedido (por exemplo, tipo de incidência),
      "Status": estado do pedido com o ID IDpedido1 (CM, CA, ...),
      "Order Control": order control do pedido (NW, RE, CA, ...),
      "Data de transacao": data de transação do pedido,
      "OBR": Campo OBR da mensagem HL7,
      "OBX": Campo OBX da mensagem HL7
    }
    IDpedido2: { ... }
  }
  PID2 : { ... }
}
```

Figura 2- Estrutura de armazenamento de dados.

De forma a facilitar o acesso aos pedidos de um determinado paciente, foi utilizado um identificador único de paciente (PID) como chave principal do dicionário acima apresentado.

Posteriormente, os valores associados ao PID contêm um outro dicionário com as informações pessoais associadas ao paciente em questão, nomeadamente o nome, a data de nascimento e género.

Adicionalmente, possui ainda todos os pedidos realizados pelo paciente. Assim, cada pedido é identificado por um ID de pedido único, facilitando a referência e o acesso a detalhes específicos. Seguidamente, dentro de cada pedido, encontram-se informações como o tipo de pedido (por exemplo, exame médico), o *status* do pedido, a data da transação, o *order control* do pedido (como NW para novo pedido ou CA para cancelado) e mais detalhes do pedido, como a descrição do exame (OBR) e relatório (OBX).

AIDA.py

Ao ser inicializado, o programa AIDA.py abre o ficheiro “AIDA.json” e carrega para um dicionário “ficheiros”, toda a informação presente nesse ficheiro. Este passo é fundamental para garantir que são carregadas as mensagens HL7 já trocadas anteriormente. Deve notar-se que o ficheiro “AIDA.json” armazena a informação relativa às mensagens da seguinte forma através da estrutura especificada anteriormente.

Para facilitar a utilização das funcionalidades do programa foi desenvolvido um menu com as seguintes opções:

- Fazer pedido
- Cancelar o pedido
- Ver pedido
- Ver todos os pedidos
- Atualizar
- Sair

A primeira opção permite fazer um pedido de exame. Para isso é necessário introduzir o id do paciente, o OBR (*observation request*) com informações específicas adicionais e, por fim, o tipo de exame pretendido. É então utilizada a função *fzrpedido*, e introduzidos os dados como se pode ver na Figura 3.

```
1
Patient ID: 123
Observation request: Pe^Partido
Tipo de exame: Rx
Mensagem enviada para PACS com sucesso!
```

Figura 3- Registo de Pedido no programa AIDA.py.

Esta função verifica se o id do paciente existe, e em caso positivo, pede ao utilizador a introdução das informações OBR e tipo de exame. No caso do utilizador não existir na base de dados, é também pedido

ao utilizador o nome, data de nascimento e género, para a criação do seu perfil e posterior inserção no dicionário JSON.

Após esta verificação, é criada a mensagem em formato *hl7* que vai ser enviada para o serviço PACS, através da sua escrita na pasta A1. Para guardar o estado do pedido, este é guardado na estrutura JSON definida anteriormente. É de notar que o pedido é guardado com estado “NW”, correspondente a um novo pedido ou pedido pendente.

De forma a verificar se o pedido foi efetuado, pode ser utilizada a funcionalidade “Ver todos os Pedidos”, que utiliza a função *listarpedidos*. Esta função é muito simples e apenas corre o ficheiro JSON onde estão armazenadas as informações dos pedidos de cada paciente.

```
4
Pedidos do paciente 123
{'Nome': 'martins' 'monica', 'Data de nascimento': '20020227', 'Genero': 'F', 'Pedidos': {'897189': {'Tipo de pedido': 'RX', 'Status': '', 'Order Control': 'NW', 'Data de transacao': '2024-03-30T20:31:07.118481', 'OBR': 'incidencia', 'OBX': ''}, '877774': {'Tipo de pedido': 'incidencia', 'Status': 'CM', 'Data de transacao': '2024-03-30T20:31:52.455582', 'Order Control': 'SC', 'OBR': '\nOBR|01|877774|877774|incidencia|'|||'|'|||'|CR|incidencia|'|||'|'^^2024-03-30T20:31:52.455582^^', 'OBX': ''}, '801671': {'Tipo de pedido': 'Rx', 'Status': '', 'Order Control': 'NW', 'Data de transacao': '2024-04-15T09:18:54.043436', 'OBR': 'Pe^Partido', 'OBX': ''}}}
```

Figura 4- Listagem de todos os Pedidos no programa AIDA.py.

Para cancelar um pedido pode ser usada a funcionalidade “Cancelar Pedido”, que utiliza a função *cancelarpedido*. A função *cancelarpedido* tem como parâmetros de entrada o id do paciente e o id do pedido que se pretende cancelar.

```
2
Introduza o ID do paciente: 123
Introduza o ID do pedido que pretende cancelar: 897189
Pedido cancelado e notificação enviada para PACS!
```

Figura 5- Cancelamento de um Pedido no programa AIDA.py.

Para cancelar um pedido, é necessário aceder ao pedido na estrutura de dados em JSON através dos ids fornecidos pelo utilizador. A função após procurar e encontrar o pedido muda o seu “Order Control” para “CA”, significativo de pedido cancelado. Na figura seguinte, mostra-se a verificação do cancelamento do pedido através da funcionalidade “Ver Pedido”.

```
3
Introduza o ID do paciente: 123
Introduza o ID do pedido: 897189
{'Tipo de pedido': 'RX', 'Status': 'CA', 'Order Control': 'CA', 'Data de transacao': '2024-04-15T09:28:06.432152', 'OBR': 'incidencia', 'OBX': ''}
```

Figura 6- Consulta de um Pedido no programa AIDA.py.

A invocação deste método faz uma procura no dicionário JSON, retornando o pedido pretendido.

Por fim, existe também a função *atualizar* associada à funcionalidade número 5. Esta funcionalidade serve para atualizar a estrutura de dados dos pedidos, verificando se existem pedidos ou alterações nos

mesmos vindos do serviço PACS. A função percorre os ficheiros da pasta A2, lendo-os e carregando-os para o ficheiro JSON.

É de notar que a script AIDA.py utiliza algumas funções auxiliares *guardarpedido* e *processa_mensagem* de forma a poder guardar os pedidos realizados e processar a mensagem vinda do serviço PACS, atualizando o dicionário de pedidos.

Quando se finaliza a utilização do serviço AIDA.py, isto é, no fim do ciclo menu, é guardado o estado do programa na esturutra mencionada acima. Quando se reinicia o programa, este estado é carregado e todos os pedidos estão atualizados.

PACS.py

Relativamente ao programa PACS.py, este começa por carregar as informações presentes na base de dados JSON gerada pelo programa AIDA para um dicionário, de modo a permitir ter acesso ao historial dos utentes presentes.

Em seguida é apresentado um menu ao utilizador, permitindo escolher uma das seguintes opções:

- Ver pedido pendentes;
- Realizacao Exame;
- Realizacao Relatorio;
- Cancelar Pedido;
- Listar pedidos;
- Atualizar;
- Sair.

Caso a opção seleccionada seja ver pedidos pendentes, é inserido o id do utente cuja informação se deseja consultar, sendo efetuada a procura dos pedidos desse utente no dicionário gerado onde o estado deste seja diferente de “CA” e “CM”, visto que estes representam pedidos cancelados e pedidos de alteração de dados, respetivamente. Assim, é retornado um dicionário contendo todos os pedidos pendentes do utente em questão, seguido da informação do término do carregamento de dados, tal como pode ser observado na Figura 7.

```
ID do paciente: 9999
A carregar pedidos pendentes de maria^josefina
{'Tipo de pedido': 'tac', 'Status': '', 'Order Control': 'NW', 'Data de transacao': '2024-04-11T11:13:47.732890', 'OBR': 'cansaco', 'OBX': ''}
Carregamento terminado!
```

Figura 7- Ver pedidos pendentes no programa PACS.py.

Ao escolher a opção de realização de exames, é solicitado ao utilizador o id do utente e do pedido associado ao exame a realizar, que se não constarem no dicionário da base de dados, são inseridos no mesmo. Tendo estes dados configurados ou recuperados, é inserida a informação relativa ao exame, nomeadamente o pedido de observação (OBR) e a data do exame e atualizado o estado do pedido para SC (Figura 8). Por fim, esta mensagem é guardada numa pasta (B2), sendo reencaminhada pelos canais Mirth para a estrutura AIDA (Figura 9).

1	MSH ^~\& AIDA AIDA PACS PACS 20240411111347732	1	MSH ^~\& PACS PACS AIDA AIDA 20240411113906242
2	ORM^O01 202404111113477325751000002533 P 2.5 AL	2	ORM^O01 202404111139062425751000002533 P 2.5 AL
3	PID 9999 maria^josefina	3	PID 9999 maria^josefina
4	20000912 F	4	20000912 F
5	PV1 I INT	5	PV1 I INT
6	15002727	6	15002727
7	ORC NW 169919 169919	7	ORC SC 169919 169919 CM
8	2024-04-11T11:13:47.732890	8	2024-04-11T11:39:06.242066
9	OBR 01 169919 169919	9	OBR 01 169919 169919 sem sinal de
10	cansaco CR tac	10	patologia CR
11	^^2024-04-11T11:13:47.732890^^	11	tac ^^2024-04-11T11:39:06.242066^^
12	0	12	0

Figura 8- Alterações ao pedido na realização de exame (PACS.py).

```
ID do pedido: 169919
Patient ID: 9999
Observation request: sem sinal de patologia
Mensagem enviada para AIDA com sucesso!
Pedido guardado!
A mensagem HL7 de realização de exame foi escrita no arquivo: B2/20240411113906242.hl7
```

Figura 9- Realização de exame no programa PACS.py.

Quanto à função de realizar relatório, esta permite, recebendo informações relativas ao id do pedido em questão, assim como do utente associado, inserir o texto a constar em relatório, identificando-se que o mesmo se poderá encontrar num PDF específico indicado (Figura 10). Esta informação é inserida na mensagem num novo campo relativo às observações (OBX), podendo ser, posteriormente, adicionadas novas informações relativas a este exame, através da adição de novos OBX, numerados sequencialmente. Adicionalmente, é importante denotar que o estado do pedido passa para realizado, RE. Aquando do término deste processo, a mensagem é guardada e enviada para a estrutura AIDA (Figura 11).

1	MSH ^~\& AIDA AIDA PACS PACS 20240411111347732	1	MSH ^~\& PACS PACS AIDA AIDA 20240411163123715
2	ORM^O01 202404111113477325751000002533 P 2.5 AL	2	ORM^O01 202404111631237155751000002533 P 2.5 AL
3	PID 9999 maria^josefina	3	PID 9999 maria^josefina
4	20000912 F	4	20000912 F
5	PV1 I INT	5	PV1 I INT
6	15002727	6	15002727
7	ORC NW 169919 169919	7	ORC RE 169919 169919 CM
8	2024-04-11T11:13:47.732890	8	2024-04-11T16:31:23.715594
9	OBR 01 169919 169919	9	OBR 01 169919 169919
10	cansaco CR tac	10	cansaco CR tac
11	^^2024-04-11T11:13:47.732890^^	11	^^2024-04-11T16:31:23.715594^^
12	0	12	OBX 1 PDF_BASE64 não se
		13	identificaram patologias
		14	visíveis 20150603012318

Figura 10- Alterações à mensagem hl7 após a realização de relatório.

```

ID do pedido: 169919
Patient ID: 9999
Escreva o relatório: não se identificaram patologias visíveis
Mensagem enviada para AIDA com sucesso!
Pedido guardado!
A mensagem HL7 de realização de exame foi escrita no arquivo: B2/20240411163123715.hl7

```

Figura 11- Realização de relatório no programa PACS.py.

Em relação à opção de cancelar pedidos, esta é igual à apresentada no programa AIDA.

```

{'Nome': 'martins^monica', 'Data de nascimento': '20020227', 'Genero': 'F', 'Pedidos': {'897189': {'Tipo de pedido': 'incidencia', 'Status': '', 'Data de transacao': '2024-03-30T20:31:07.118481', 'Order Control': 'NW', 'OBR': '\nOBR|01|897189|897189|incidencia|^^^^2024-03-30T20:31:07.118481^^', 'OBX': ''}, '877774': {'Tipo de pedido': 'incidencia', 'Status': '', 'Data de transacao': '2024-03-30T20:28:22.765296', 'Order Control': 'NW', 'OBR': '\nOBR|01|877774|877774|incidencia|^^^^2024-03-30T20:28:22.765296^^', 'OBX': ''}}}
Pedidos do paciente 1234
{'Nome': 'xonc', 'Data de nascimento': '12341234', 'Genero': 'F', 'Pedidos': {'189162': {'Tipo de pedido': '32', 'Status': '', 'Data de transacao': '2024-04-02T13:27:41.271365', 'Order Control': 'NW', 'OBR': '\nOBR|01|189162|189162|32|^^^^2024-04-02T13:27:41.271365^^', 'OBX': ''}}}
Pedidos do paciente 9999
{'Nome': 'maria^josefina', 'Data de nascimento': '20000912', 'Genero': 'F', 'Pedidos': {'169919': {'Tipo de pedido': 'cansaco', 'Status': '', 'Data de transacao': '2024-04-11T11:13:47.732890', 'Order Control': 'NW', 'OBR': '\nOBR|01|169919|169919|cansaco|^^^^2024-04-11T11:13:47.732890^^', 'OBX': ''}}}
Todos os pedidos foram listados

```

Figura 12- Listagem de pedidos no programa PACS.py.

Adicionalmente, surge a opção de listar pedidos, a partir da qual surge uma listagem de todos os pedidos efetuados, recuperados através da estrutura JSON carregada e atualizada.

Por fim, pode-se, ainda, atualizar a estrutura de dados JSON, Figura 13, mantendo todos os registos efetuados, mesmo após o encerramento do programa em questão, garantindo a propriedade da durabilidade.

```

{
  "9999": {
    "Nome": "maria^josefina",
    "Data de nascimento": "20000912",
    "Genero": "F",
    "Pedidos": {
      "169919": {
        "Tipo de pedido": "cansaco",
        "Status": "",
        "Data de transacao": "2024-04-11T11:13:47.732890",
        "Order Control": "NW",
        "OBR": "\nOBR|01|169919|169919|cansaco|^^^^2024-04-11T11:13:47.732890^^",
        "OBX": ""
      }
    }
  }
}

```

Figura 13- Atualização da estrutura JSON.

Assim, obtém-se um modo simples e eficaz de edição e transmissão de informação, permitindo que os dados sejam sempre mantidos para posterior consulta ou alteração.

Conclusão

Durante a realização deste trabalho foram encontrados alguns obstáculos e dificuldades, nomeadamente a compreensão da estrutura das mensagens HL7, bem como o significado de cada um dos seus campos. Para além disso, foram encontrados alguns problemas durante a configuração dos canais no Mirth Connect. No entanto, estes obstáculos foram ultrapassados com sucesso, o que resultou num sistema eficiente para a troca de informações entre os serviços AIDA e PACS, simulando assim a interação entre Serviços de Informação Hospitalares.

O trabalho desenvolvido proporcionou uma oportunidade para aprofundar os conhecimentos abordados na Unidade Curricular de Aplicações Informáticas em Engenharia Biomédica, nomeadamente uma melhor compreensão da estrutura de mensagens HL7 e a sua importância para garantir a interoperabilidade entre sistemas de saúde. Para além disso, foi possível aprimorar a utilização do Mirth Connect através da construção de canais que permitiam o envio e a receção de mensagens entre serviços. O desenvolvimento dos canais Python foi fundamental para aprimorar a utilização desta linguagem, principalmente no que diz respeito à manipulação de ficheiros e estruturas de dados.

É importante referir que, apesar dos objetivos propostos terem sido cumpridos com sucesso, existem melhorias que poderiam ser implementadas no futuro, de modo a aproximar ainda mais este sistema a uma utilização no mundo real. Trabalho futuro poderia passar pela inclusão de mensagens HL7 relativas a outros eventos, como é o caso de mensagens ADT (Admit, Discharge and Transfer) ou mensagens OML (relacionadas com pedidos laboratoriais). Para além disso, também seria pertinente efetuar o tratamento de outros segmentos de mensagens, como por exemplo, o segmento PV1 (Patient Visit).